

1. PROJECT OVERVIEW

- **Project Title:** FitBuddy - AI Workout Generator

2. INTRODUCTION TO THE PLANNING PHASE:

The Planning Phase establishes the architectural blueprint for FitBuddy. Developing a personalized health-tech application requires seamless synchronization between frontend data collection, backend database management (SQLite), and external cloud AI generation (Google Gemini). A structured roadmap was essential to manage these moving parts and ensure rapid deployment.

3. ITERATIVE DEVELOPMENT STRATEGY

The project utilized an iterative, milestone-based development lifecycle. This approach allowed the team to construct and validate the database schema before connecting the external AI model, ensuring that data persistence was stable before handling dynamic content generation.

4. MILESTONE MAPPING & EXECUTION

Milestone 1: Environment Setup & Architecture

- **Primary Goal:** Initialize the local development workspace and finalize the technology stack.
- **Key Activities:** Configured the Python 3 virtual environment. Installed core dependencies including FastAPI, SQLAlchemy, and `google-generativeai`. Established the project directory structure separating routing, database models, and HTML templates.

Milestone 2: Database Modeling (SQLAlchemy)

- **Primary Goal:** Design the relational database schema to persistently store user metrics and AI outputs.
- **Key Activities:** Authored `database.py` to create the SQLite database. Defined the `User` class (capturing name, age, weight, goal, intensity) and the

WorkoutPlan class using SQLAlchemy's declarative base to map Python objects to SQL tables.

Milestone 3: AI Integration & Backend Routing

- **Primary Goal:** Connect the backend logic to the Google Gemini model.
- **Key Activities:** Configured the `gemini-3.5-flash-lite` model in `gemini_generator.py`. Engineered the prompt structure to ingest the user's specific fitness goal and intensity. Authored `routes.py` to handle the FastAPI POST requests and orchestrate the sequence from data collection to AI generation to database committing.

Milestone 4: Frontend & Dashboard Development

- **Primary Goal:** Build the user-facing web pages and administrative interfaces.
- **Key Activities:** Designed the primary HTML input form to capture physiological metrics. Utilized Jinja2 templates to dynamically render the AI-generated 7-day workout plan back to the user. Developed the Admin Dashboard UI to display a comprehensive tabular view of all registered users and their corresponding workout plans.

Milestone 5: System Testing & Refinement

- **Primary Goal:** Validate the application's stability under execution and handle API exceptions.
- **Key Activities:** Booted the application using the `uvicorn app.main:app --reload` command. Implemented strict exception handling in the AI module, explicitly adding a 60-second timeout parameter to prevent infinite server hanging during external API latency.

5. ANTICIPATED CHALLENGES & CONTINGENCIES

- **Challenge 1: AI Latency and Server Timeouts.** Cloud AI generation can occasionally experience delays, potentially freezing the FastAPI server.

- *Contingency*: Integrated a strict `request_options={"timeout": 60}` configuration within the Gemini API call, backed by a `try-except` block to return a clean error message rather than crashing the application.
- **Challenge 2: Database Session Management.** Improper database connections can lead to locked SQLite files.
 - *Contingency*: Utilized SQLAlchemy's `SessionLocal` within the routing functions to ensure database connections are properly opened, committed, and closed for every single POST request.